An $O(n)$ Algorithm for finding a tree 3-spanner on permutation graphs

Hon-Chan Chen* and Shin-Huei Wu**

* Department of Shipping Transportation and Management

National Kaohsiung Institute of Marine Technology

Kaohsiung, Taiwan

** Department of Computer Science and Engineering

National Sun Yat-sen University

Kaohsiung, Taiwan

Abstract

A tree 3-spanner of a graph G is a spanning tree T of G, in which the distance between any two vertices in T is at most 3 times their distance in G. Madanlal et al. presented an $O(n + m)$ time algorithm for finding a tree 3-spanner of a permutation graph [6]. In this paper, we will present an improved algorithm to solve the same problem in $O(n)$ time.

1. Introduction

Let $G = (V, E)$ be a connected graph with vertex set V and edge set E. A graph $H = (V', E')$ is a *spanning subgraph* of G if $V' = V$ and $E' \subseteq E$. If H is an acyclic connected

spanning subgraph, then H is a *spanning tree* of G. A spanning tree T of G is a *tree t-spanner* if the distance between any two vertices in T is at most t times their distance in G, where t is a positive integer. The concept of tree spanner has applications in reducing the cost of a network. Beside that, it can be used in distributed environments [7].

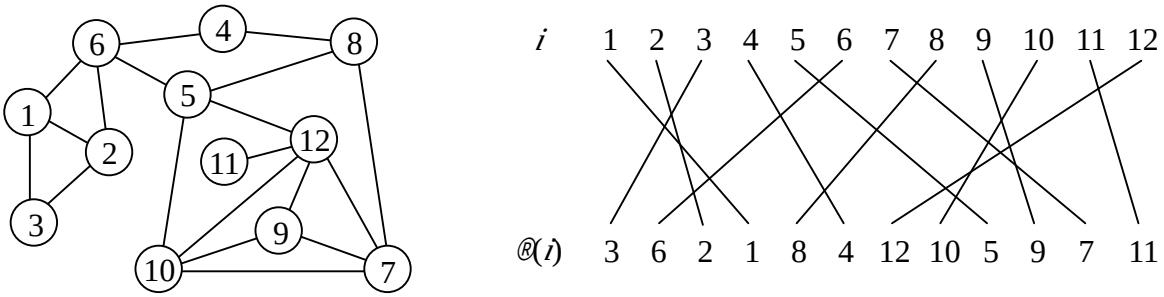
Cai and Corneil in [1] showed that the problem of deciding the existence of t-spanner on unweighted graphs is NP-complete when $t \geq 4$, and it can be solved in polynomial time when $t \leq 2$. The case where $t = 3$ is an open problem; however, it is conjectured by Cai to be NP-complete [2]. Many researchers have studied the problem of tree spanner on special types of graphs. For example, Madanlal et al. presented algorithms of finding tree 3-spanners on interval, permutation, and regular bipartite graphs [6]. The spanner problem on the hypercube and bounded degree graphs were discussed in [4] and [3], respectively.

In [6], a tree 3-spanner of a permutation graph can be found in $O(n + m)$ time, where n is the number of vertices and m is the number of edges. In this paper, we will present an improved algorithm to solve the same problem in $O(n)$ time, assuming the input permutation graph is connected. The rest of this paper is organized as follows. In Section 2, we first introduce the permutation graph and then introduce Madanlal's algorithm of finding a tree 3-spanner of a permutation graph. In Section 3, we present our $O(n)$ algorithm and show its correctness. Finally, we give concluding remarks in Section 4.

2. Review

Before introducing Madandal's algorithm, we introduce the definition of permutation graphs [5]. Let π be a permutation sequence of the numbers 1, 2, …, n. Moreover, let $\pi^{-1}(i)$ be the position in π sequence where the number i can be found from left to right. Then, permutation graph $G = (V, E)$ has vertex set $V = \{1, 2, \ldots, n\}$ and $(i, j) \in E$ if and only if (i –

$j)(\pi^{-1}(i) - \pi^{-1}(j)) < 0$. A permutation graph can be represented by a permutation diagram. In the diagram, there are two sequences 1, 2, …, n and $\pi(1)$, $\pi(2)$, …, $\pi(n)$. Each vertex i in G corresponds to a line between i and $\pi^{-1}(i)$ in the diagram. Two vertices are joined by an edge in G if and only if the corresponding two lines intersect in the diagram. For example, in Figure 1, $\pi(1) = 3$, $\pi^{-1}(1) = 4$, and $(1, 6) \in E$. From the definition, we know that for three vertices i, j, and k, $i < j < k$, if i is adjacent to k, then j is adjacent to i or k.



(a) (b)

Figure 1. (a) A permutation graph. (b) The corresponding diagram.

Madandal's algorithm constructs a tree 3-spanner T in several stages. Let $A_i = \{j \mid j > i,$ j is adjacent to $i\}$ for $1 \le i \le n$. Define

$$P_{\max}(i) = \begin{cases} \max A_i & \text{, if } A_i \neq \varnothing, \\ i & \text{, otherwise.} \end{cases}$$

In the first stage, let $\sim_1 = P_{\max}(i)$, and consider all the vertices in interval $[1, \sim_1]$. In subsequent stages, consider all the vertices in the intervals of the form $(\sim_{i-1}, \sim_i]$, where each of the $\sim_i$s is defined when the algorithm proceeds. For each interval $(\sim_{i-1}, \sim_i]$, identify a special vertex $r_i \in (\sim_{i-2}, \sim_{i-1}]$ such that r_i is adjacent to $\sim_i$. The edges for the vertices in $(\sim_{i-1}, \sim_i]$ are included in T carefully starting from the inclusion of the edge $(r_i, \sim_i)$. The following is Madandal's algorithm.

Algorithm A

Input: A connected permutation graph with the permutation numbering

Output: Tree 3-spanner T

Methods:

Step 1. Let $i = 1$, $r_i = 1$, $\sim_0 = 1$, and $\sim_1 = P_{\max}(1)$.

Step 2. While $\sim_i \leq n$, do the following substeps.

2.1. Add edge $(r_i, \sim_i)$ to T. /* Begin stage i. */

2.2. Let $S_i = \{v \mid v \in (\sim_{i-1}, \sim_i)$ and v is adjacent to $r_i\}$ and $R_i = \{v \mid v \in (\sim_{i-1}, \sim_i)$ and v is not adjacent to $r_i\}$.

2.3. For all $v \in S_i$, add edge (v, r_i) to T. Moreover, for all $v \in R_i$, add edge $(v, \sim_i)$ to T. /* Since $r_i < v < \sim_i$ and r_i is adjacent to $\sim_i$, v is adjacent to r_i or $\sim_i$. */

2.4. If $\sim_i = n$, then stop Algorithm A. /* All vertices have been included in T. */

2.5. Let $\sim_{i+1} = \max_{v \in R_i} P_{\max}(v)$. /* Define $\sim_{i+1}$ for the next stage. */

2.6. Let $T_i = \{v \mid v \in R_i$ and v is adjacent to $\sim_{i+1}\}$.

2.7. Choose r_{i+1} from T_i such that if for some $v \in (\sim_i, \sim_{i+1})$ and v is not adjacent to r_{i+1}, then q is not adjacent to v for all $q \in T_i$. /* Define r_{i+1} for the next stage. */

2.8. Let $i = i + 1$. /* Enter next stage. */

End of Algorithm

We use Figure 1 to illustrate the above algorithm. In stage 1, $r_1 = 1$, $\sim_1 = P_{\max}(1) = 6$, $S_1 = \{2, 3\}$, and $R_1 = \{4, 5\}$. Moreover, $\sim_2 = \max\{P_{\max}(4), P_{\max}(5)\} = \max\{8, 12\} = 12$, $T_1 = \{5\}$, and $r_2 = 5$. In stage 2, $S_2 = \{8, 10\}$ and $R_2 = \{7, 9, 11\}$. Since $\sim_2 = 12$, we stop this algorithm. The resulting tree 3-spanner is shown in Figure 2.

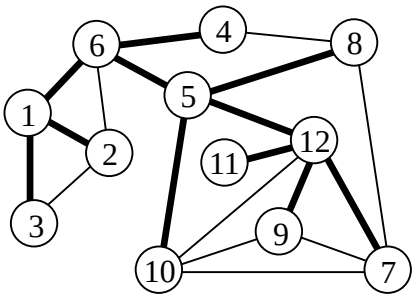


Figure 2. The resulting tree 3-spanner.

In Algorithm A, we need to determine $\sim_{i+1} = \max_{v \in R_i} P_{\max}(v)$ in each stage. Since finding $P_{\max}$ for all vertices takes $O(m)$ time [6], the complexity of Madandal's algorithm is $O(n + m)$ even when the permutation diagram is given.

3. An $O(n)$ Algorithm for Finding a Tree 3-Spanner

In this section, we shall present an $O(n)$ algorithm for finding a tree 3-spanner of a permutation graph, which follows Madandal's main idea. Let $L(i) = \max\{\pi(1), \pi(2), ..., \pi(i)\}$ and $U(i) = \max\{\pi^{-1}(1), \pi^{-1}(2), ..., \pi^{-1}(i)\}$, $1 \le i \le n$. Denote $M[i]$ the set of vertices adjacent to i and i itself. We can observe that the maximal vertex in $M[i]$ is $L(\pi^{-1}(i))$. Moreover, the vertex in $M[i]$ which appears at the most right position on π sequence is $\pi(U(i))$. With the above concept, we can easily find a tree 3-spanner. The following is our algorithm, in which notation is the same to that in Algorithm A.

Algorithm B

Input: A connected permutation graph with π sequence

Output: Tree 3-spanner T

Methods:

Step 1. Let $L(i) = \max\{\mathcal{R}(1), \mathcal{R}(2), ..., \mathcal{R}(i)\}$ and $U(i) = \max\{\mathcal{R}^{-1}(1), \mathcal{R}^{-1}(2), ..., \mathcal{R}^{-1}(i)\}$ for $i = 1, 2, ..., n$.

Step 2. Let $i = 1$, $r_i = 1$, $\sim_0 = 1$, and $\sim_1 = L(\mathcal{R}^{-1}(1))$.

Step 3. While $\sim_i \leq n$, do the following substeps.

3.1. Add edge $(r_i, \sim_i)$ to T. /* Begin stage i. */

3.2. For each vertex v in $(\sim_{i-1}, \sim_i)$, if (v, r_i) is an edge, then add (v, r_i) to T; otherwise, add edge $(v, \sim_i)$ to T. /* Vertex v is adjacent to r_i or $\sim_i$. */

3.3. If $\sim_i = n$, then stop Algorithm B. /* All vertices have been included in T. */

3.4. Let $\sim_{i+1} = L(U(\sim_i))$ and $r_{i+1} = \mathcal{R}(U(\sim_i))$. /* Define $\sim_{i+1}$ and r_{i+1}. */

3.5. Let $i = i + 1$. /* Enter next stage. */

End of Algorithm

We take the example of Figure 1 to illustrate Algorithm B. The values of $\mathcal{R}(i)$, $\mathcal{R}^{-1}(i)$, $L(i)$, and $U(i)$, $i = 1, 2, .., 12$, is shown in Table 1. In stage 1, $r_1 = 1$ and $\sim_1 = L(\mathcal{R}^{-1}(1)) = L(4) = 6$. Vertices 2 and 3 are adjacent to r_1 while vertices 4 and 5 are not. Moreover, $\sim_2 = L(U(\sim_1)) = L(U(6)) = L(9) = 12$ and $r_2 = \mathcal{R}(U(\sim_1)) = \mathcal{R}(9) = 5$. In stage 2, vertices 8 and 10 are adjacent to r_2 while vertices 7, 9, and 11 are not. Since $\sim_2 = 12$, we stop this algorithm. The resulting tree 3-spanner is the same to that in Figure 2.

i	1	2	3	4	5	6	7	8	9	10	11	12
$\mathcal{R}(i)$	3	6	2	1	8	4	12	10	5	9	7	11
$\mathcal{R}^{-1}(i)$	4	3	1	6	9	2	11	5	10	8	12	7
$L(i)$	3	6	6	6	8	8	12	12	12	12	12	12
$U(i)$	4	4	4	6	9	9	11	11	11	11	12	12

Table 1. The values of $\mathcal{R}(i)$, $\mathcal{R}^{-1}(i)$, $L(i)$, and $U(i)$.

The correctness of Algorithm B can be proved by the following lemmas.

Lemma 3.1. *For any vertex i, $1 \le i \le n$, $P_{\max}(i) = L(\mathcal{R}^{-1}(i))$.*

Proof. By the definition of $P_{\max}(i)$, we have $\mathcal{R}^{-1}(P_{\max}(i)) \le \mathcal{R}^{-1}(i)$. Since $L(\mathcal{R}^{-1}(i)) = \max\{\mathcal{R}(1), \mathcal{R}(2), \ldots, \mathcal{R}(\mathcal{R}^{-1}(i))\}$, $L(\mathcal{R}^{-1}(i)) = P_{\max}(i)$. Assume to the contrary that $L(\mathcal{R}^{-1}(i)) = j > P_{\max}(i) \ge i$ for some j. Then, $\mathcal{R}^{-1}(j) < \mathcal{R}^{-1}(i)$ and j is adjacent to i. It contradicts the definition of $P_{\max}(i)$. Therefore, $P_{\max}(i) = L(\mathcal{R}^{-1}(i))$.

Q.E.D.

Lemma 3.2. *Each $\sim_i$, $1 \le i \le k$, defined in Algorithm A is equal to $L(U(\sim_{i-1}))$.*

Proof. Initially, $\sim_0 = 1$, and by Lemma 3.1, $\sim_1 = P_{\max}(1) = L(\mathcal{R}^{-1}(1)) = L(U(1))$. In Algorithm A, we define $\sim_{i+1} = \max_{v \in R_i} P_{\max}(v)$, where $R_i = \{j \mid j \in (\sim_{i-1}, \sim_i)$ and j is adjacent to $\sim_i\}$. Since $P_{\max}(v) = L(\mathcal{R}^{-1}(v))$, if $\mathcal{R}^{-1}(u) > \mathcal{R}^{-1}(v)$ for any two vertices u and v, then $P_{\max}(u) \ge P_{\max}(v)$. Thus, $\sim_{i+1} = P_{\max}(u)$ for some $u \in R_i$ with $\mathcal{R}^{-1}(u) = \max\{\mathcal{R}^{-1}(j) \mid j \in R_i\}$. Since any vertex in R_i is smaller than $\sim_i$, $\mathcal{R}^{-1}(u) = \max\{\mathcal{R}^{-1}(1), \mathcal{R}^{-1}(2), \ldots, \mathcal{R}^{-1}(\sim_i)\} = U(\sim_i)$. We obtain $\sim_{i+1} = P_{\max}(u) = L(\mathcal{R}^{-1}(u)) = L(U(\sim_i))$.

Q.E.D.

Lemma 3.3. *Each r_i, $1 \le i \le k$, defined in Algorithm A can be substituted by $\mathcal{R}(U(\sim_{i-1}))$.*

Proof. It is true that $r_1 = 1 = \mathcal{R}(\mathcal{R}^{-1}(1)) = \mathcal{R}(U(1)) = \mathcal{R}(U(\sim_0))$. In Algorithm A, we choose r_{i+1} from T_i, where $T_i = \{j \mid j \in R_i$ and j is adjacent to $\sim_{i+1}\}$, such that if for some $v \in (\sim_i, \sim_{i+1})$ and v is not adjacent to r_{i+1}, then q is not adjacent to v for all $q \in T_i$. The simplest way to choose r_{i+1} is choosing the vertex u from T_i with $\mathcal{R}^{-1}(u) = \max\{\mathcal{R}^{-1}(q) \mid q \in T_i\}$. The reason

is that if u is not adjacent to any vertex $v \in (\sim_i, \sim_{i+1})$, then for any vertex $q \in T_i$, $q \neq u$, we have $q < v$ and $\mathcal{R}^{-1}(q) < \mathcal{R}^{-1}(u) < \mathcal{R}^{-1}(v)$. Thus, q is not adjacent to v. Since all vertices of T_i are smaller than $\sim_i$, $\mathcal{R}^{-1}(u) = \max\{\mathcal{R}^{-1}(1), \mathcal{R}^{-1}(2), \ldots, \mathcal{R}^{-1}(\sim_i)\} = U(\sim_i)$. Therefore, $u = \mathcal{R}(\mathcal{R}^{-1}(u)) = \mathcal{R}(U(\sim_i))$. In Algorithm A, there may be more than one vertex which can be r_{i+1}. However, $u = \mathcal{R}(U(\sim_i))$ must be able to be r_{i+1}. We claim that r_{i+1} found in Algorithm A can be substituted by $\mathcal{R}(U(\sim_i))$.

Q.E.D.

Theorem 3.4. *Algorithm B finds a tree 3-spanner of a permutation graph in $O(n)$ time.*

Proof. By the above lemmas, we know r_i and $\sim_i$, $1 \le i \le k$, defined in Algorithm A can be substituted by our r_i and $\sim_i$ defined in Algorithm B. For all vertices v in each interval $(\sim_{i-1}, \sim_i)$, our algorithm uses the same manner in Algorithm A to add edge (v, r_i) or $(v, \sim_i)$ to T. Therefore, the resulting tree found by Algorithm B is a tree 3-spanner of a permutation graph. The most consuming steps in Algorithm B are Step 1 and Step 3.2. To compute $L(i)$ and $U(i)$, $i = 1, 2, \ldots, n$, we can scan $\mathcal{R}$ and $\mathcal{R}^{-1}$ sequences from left to right and keep each latest maximal value during the scanning. Thus, $L(i)$ and $U(i)$ can be computed in $O(n)$ time. Suppose the input permutation graph G needs k stages of Algorithm B. In Step 3.2, we visit each vertex in $(\sim_{i-1}, \sim_i)$ once in stage i, $1 \le i \le k$. It is trivial that Step 3.2 takes $O(n)$ time totally after k stages. Thus, the complexity of Algorithm B is $O(n)$.

Q.E.D.

4. Concluding Remarks

In [6], Madanlal et al. presented an $O(n + m)$ algorithm for finding a tree 3-spanner of a

permutation graph. It is inefficient when the π sequence is given. In this paper, we present an improve algorithm so that the same problem can be solved in $O(n)$ time. Madanlal et al. also presented an algorithm for tree 3-spanners on interval graphs. Since the classes of interval graphs and permutation graphs are subclasses of trapezoid graphs, there may exist a polynomial time algorithm of finding a tree 3-spanner of a trapezoid graph. It is interesting and worthy to extend algorithms for trapezoid graphs.

References

[1] Leizhen Cai and D.G. Corneil, Tree Spanners, *SIAM Journal on Discrete Mathematics*, Vol. 8, 1995, pp. 359-387.

[2] Leizhen Cai, Tree Spanners: Spanning Trees that Approximate Distances, *Technique Reprots 260/92*, Department of Computer Science, University of Toronto.

[3] Leizhen Cai and Mark keil, Spanners in Graph of Bounded Degree, *Networks*, Vol. 24, 1994, pp. 233-249

[4] W. Duckworth and M. Zito, Sparse Hypercube 3-Spanners, *Discrete Applied Mathematics,* Vol. 103, 2000, pp. 289-295.

[5] M. C. Golumbic, *Algorithmic Graph Theory and Perfect Graphs*, Academic Press, 1980.

[6] M. S. Madanlal, G. Venkatesan, C. Pandu Rangan, Tree 3-Spanners on Interval, Permutation and Regular Bipartite Graphs, *Information Processing Letters*, Vol. 59, 1996, pp. 97-102.

[7] D. Peleg and J. D. Ullman, An Optimal Synchronizer for the Hypercube, in: *Proceeding of the 6th ACM Symposium on Principles of Distributed Computing*, 1987, pp. 77-85.